



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Készítette:	Horváth Gellért
Neptun-kód:	BFK4J4
Szakirány:	Mérnökinformatika
E-mail cím:	gellihorvath@gmail.com
Konzulens(ek):	Pálfy Miklós Gábor
E-mail címe(ik):	palfy.miklos@vik.bme.hu

**Téma címe: Modern telefonos applikáció fejlesztése
agilis módszertan alkalmazásával**

Feladat

A félév során folytatom a szakdolgozati projektem fejlesztését: az alkalmazás egyes funkcióit újragondolom és bővítom, fejlesztem a projektkövetési szemléletemet az agilis módszertan szerint, valamint megvalósítok egy GitHub CI/CD pipeline-t, amely magában foglalja a backend automatikus buildelését és VPS szerveren történő hosztolását. A pipeline-folyamatokat igyekszem egyszerűbbé és újrafelhasználhatóvá tenni a backend konténerizálásával.

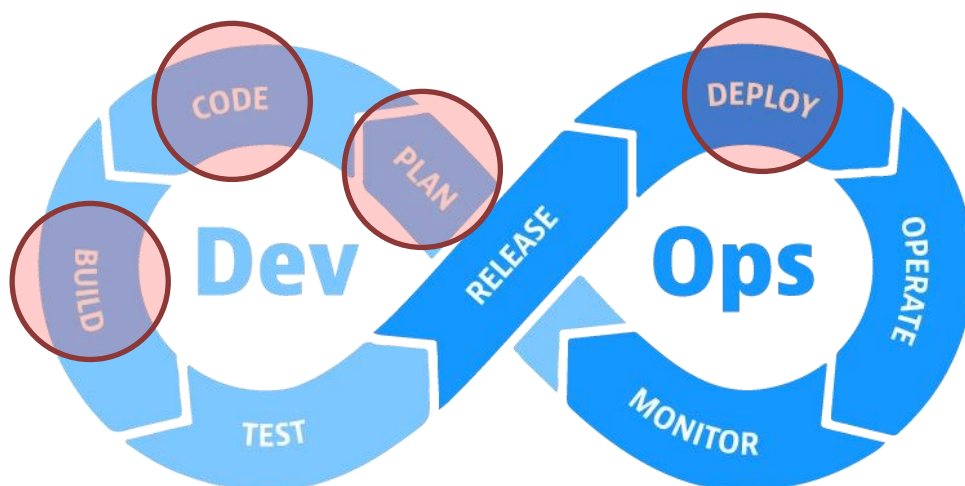
2024/2025. 2. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1 Bevezető

A félév során a szakdolgozat projektemet fejlesztettem tovább az agilis módszertanok és technikák alkalmazásával, amely sokkal rendezettebbé és nyomon követhetőbbé tette a munkámat. A szakdolgozat készítése során korábban teljesen rendszertelenül végeztem a fejlesztést: nem követtem nyomon, hogy a szoftverfejlesztési életciklus mely szakaszában tartok, illetve milyen feladataim vannak még hátra. Emiatt legtöbbször azzal foglalkoztam, ami éppen először eszembe jutott. Bár a szakdolgozatot sikeres projektnek tekinthetem, folyamatosan fel kellett vennem a fonalat azzal kapcsolatban, hogy egy-egy feladat milyen állapotban van, ami jelentősen megnövelte a fejlesztési időt.

Emellett a projekt során semmilyen DevOps-megközelítést nem alkalmaztam, ezen azonban szerettem volna változtatni annak érdekében, hogy a projekt minősége minden szempontból javuljon. A legfontosabb személyes célom az volt, hogy a backendet kitelepítsem egy nyilvánosan elérhető szerverre, így a mobilalkalmazás bármilyen hálózathoz képes legyen csatlakozni hozzá.



1. ábra DevOps életciklusok

Konzulensemmel azt a célt tűztük ki, hogy mivel a feladat a teljes fejlesztési folyamat modellezése, ezért a teljes szoftverfejlesztési életciklus bizonyos részeit emeljük ki, és ezekre fókuszáljak az önálló laboratórium során. A kiválasztott életciklusokat az 1. ábra mutatja; a pirossal bekeretezett szakaszokat kellett modelleznem. Ennek során gyakoroltam a DevOps-ciklus fejlesztési (dev) és üzemeltetési (ops) részeit is, vagyis a tervezést, az új funkciók implementálását, majd az új szoftververzió automatikus buildelését és a backend szerverre történő kitelepítését, amely egyben a személyes fő célom is volt.

1.2 A munka állapota, készültségi foka a félév elején

A félév elején a szakdolgozat projektomból indultam ki, amelynek minden része működőképes volt, és a kódbázis is jó minőségűnek bizonyult. Ugyanakkor projektszinten több hiányosság is jelen volt, amelyeket a félév végére sikerült javítanom. A projekt kezdeti állapotában minden komponens sikeresen buildelhető volt, a backend szervert lokálisan tudtam

futtatni, valamint a mobilalkalmazás is működött telefonon.

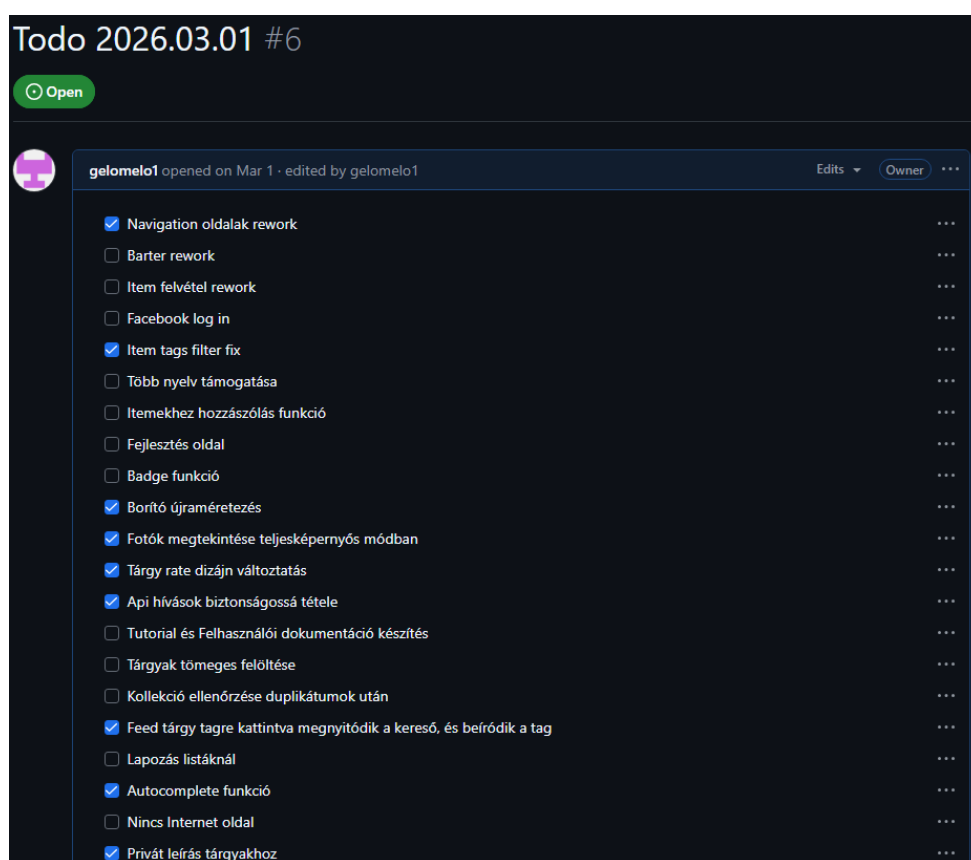
Ezen kívül azonban nem állt rendelkezésre megfelelő feladatnyilvántartás az alapvető verziózást leszámítva nem volt kialakított minőségi verziókövetési folyamat, CI/CD pipeline, továbbá a backend szerver a lokális hálózaton kívülről nem volt elérhető.

2. Az elvégzett munka és eredmények ismertetése

2.1 Plan

Először a tervezési (plan) szakasszal foglalkoztam. Ennek során arra kerestem megoldást, hogyan tudom a projekt funkcióit és az ezekhez tartozó feladatokat hatékonyan nyomon követni. Emellett az agilis módszertanokra jellemző, gyakran változó követelmények és feladatok kezelését, illetve modellezését is igyekeztem megvalósítani.

2.1.1 Feladatok követése Github Issue-ban



2. ábra Projekt feladatok GitHub Issue-ban

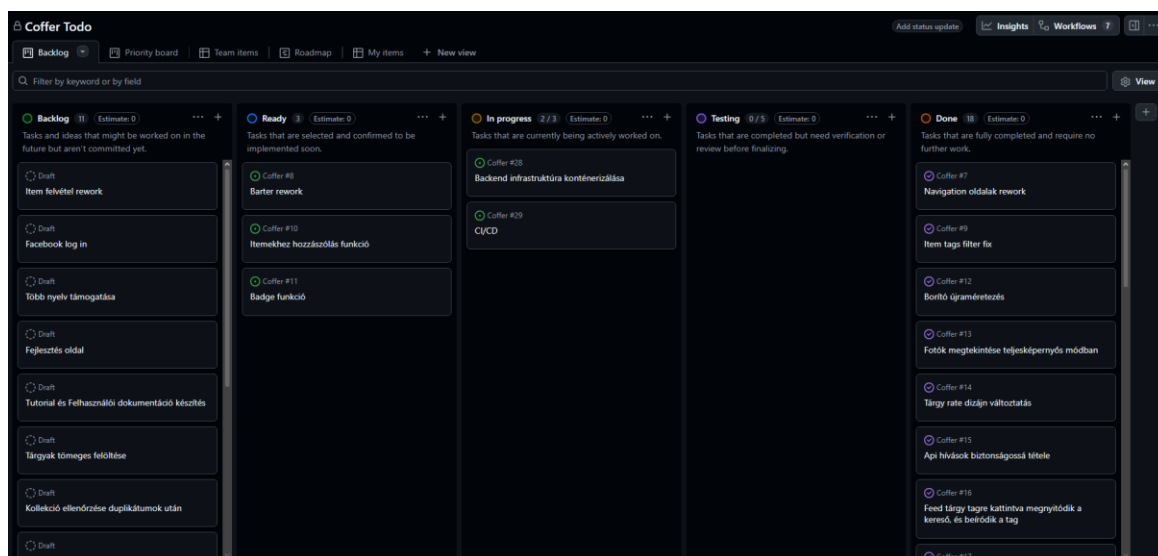
A feladatok nyomon követésére kezdetben egy nagyon egyszerű megoldást választottam: egy GitHub Issue-t használtam to-do listaként. Egyetlen issue-ba gyűjtöttem össze az összes feladatot és funkcióötletet. Ehhez a 2. ábra által mutatott GitHub Markdown checkbox funkciót alkalmaztam: minden listaelemhez Markdown segítségével hoztam létre egy jelölőnégyzetet, majd egy funkció elkészülte után azt bepípáltam.

Ez a megoldás véleményem szerint kisebb, néhány funkcióból álló projektek esetén jól működhet, mivel rendkívül egyszerű, és gyakorlatilag semmilyen külön karbantartást vagy kezelést nem igényel. Ugyanakkor üzleti vagy összetettebb projektek esetében ez a túlzott

egyszerűség már nem állná meg a helyét. Mivel az önálló laboratórium során egy professzionálisabb fejlesztési folyamat modellezése volt a célom, ezért egy fejlettebb megoldást kezdtem keresni.

2.1.2 Feladatok követése Github Projects-ben

Az agilis megközelítés általában Kanban- vagy sprinttáblákat alkalmaz a feladatok hatékony nyomon követésére. A sprintalapú megközelítést végül azért nem választottam, mert a projekten nem tudtam folyamatosan dolgozni, így célzott sprintek tervezése és végrehajtása sem lett volna hatékony. Ehelyett a Kanban-megoldás mellett döntöttem, és a GitHub Projects Kanban-tábláját kezdtem használni.



3. ábra Projekt feladatok GitHub Projects-ben

A választásom mellett több szempont is szólt: a GitHub platformot már ismertem, ingyenesen elérhető, nem szerettem volna egy újabb rendszert megtanulni és kezelni, valamint a GitHub Projects az egyszerűbb Kanban-folyamatok támogatására teljes mértékben megfelelő funkcionalitást biztosít. A 3. ábra mutatja az általam létrehozott táblát. Az alapértelmezett Kanban-felosztás szinte teljes egészében megfelelt az igényeimnek, csupán néhány kisebb módosítást végeztem rajta. Például eltávolítottam az oszlopokhoz tartozó korlátokat, illetve rövid leírásokat adtam hozzájuk annak érdekében, hogy egyértelmű legyen, melyik oszlop milyen állapotot jelöl.

A Backlog oszlopban draft elemek találhatók. Ezekhez a GitHub nem hoz létre külön issue-t; ide jegyzem fel a brainstorming során felmerülő ötleteket és lehetséges jövőbeli funkciókat. A Ready oszlop tartalmazza azokat a feladatokat, amelyeket már kiválasztottam, biztosan meg szeretnék valósítani, és hamarosan megkezdem az implementálásukat. Az In Progress oszlopban találhatók az aktuálisan fejlesztés alatt álló funkciók. A Testing oszlopba kerülnek azok a funkciók, amelyek implementálása már elkészült, azonban még ellenőrizni szükséges a helyes működésüket. Végül a Done oszlop tartalmazza a teljesen elkészült funkciókat.

2.1.3 Gyakran változó követelmények szemléletmód modellezése

Az agilis szemlélet egyik legfontosabb alapelve, hogy a fejlesztés során folyamatosan készen kell állni a követelmények változására, illetve új funkciók bevezetésére, akár a projekt későbbi szakaszaiban is. [1] Ezt a megközelítést úgy modelleztem, hogy rendszeresen

brainstorming sessionöket tartottam, amelyek során új, hasznos funkcióötleteket kerestem. Azokat az ötleteket, amelyeket megfelelőnek találtam, feljegyeztem a Kanban-táblán.

Ezzel lényegében megvalósítottam az agilis szemlélet egyik fontos részét: úgy kezeltem a projektet, mintha egy ügyfél a már nagyrészt elkészült alkalmazáshoz kérne új funkciókat. A szemlélet másik részét, vagyis a rendszer bővíthetőségére és a változások befogadására való nyitottságot, a fejlesztési és kódolási folyamat során igyekeztem megvalósítani. Erről a [2.2-es](#) fejezetben írok részletesebben.

2.2 Code

Az önálló laboratórium során a fejlesztési (coding) szakaszban több új funkciót is megvalósítottam. Ezen a területen a szakdolgozathoz képest nem jelent meg számomra lényeges újdonság, mivel már korábban is nagy hangsúlyt fektettem a megfelelő kódminőségre. A fejlesztés során továbbra is betartottam a legfontosabb szoftverfejlesztési alapelveket: strukturáltan tartottam a projektet, követtem az egyes programozási nyelvekhez tartozó formázási és jóformáltsági ajánlásokat, valamint interface-eket és különböző tervezési mintákat alkalmaztam.

Ezek az eszközök és gyakorlatok biztosítják, hogy a kód könnyen karbantartható és bővíthető maradjon. Ez különösen fontos az agilis fejlesztési szemléletben, ahol a követelmények gyakran változnak, [1] így az új funkciókat gyorsan és a lehető legkisebb módosítással kell integrálni a meglévő rendszerbe.

A fejlesztési szakasz során több jelentős funkcionalitást is kialakítottam. Az egyik legfontosabb változtatás a teljes navigációs rendszer és az oldalstruktúra újragondolása volt. A korábbi megoldásban a navigációs sáv csak egy kollekció kiválasztása után jelent meg, a kollekció kontextusában. Ez azonban nem felelt meg a modern mobilalkalmazásokban megszokott működésnek, ezért átalakítottam a rendszert úgy, hogy a kollekciófüggetlen funkciók és maga a navigációs sáv már az alkalmazás megnyitásakor elérhető legyen. A felhasználó ezt követően választhat kollekciót a kollekcióspecifikus funkciók használatához. Ez a megközelítés természetesebb és intuitívabb felhasználói élményt biztosít.

Szintén fontos fejlesztési feladat volt a backend biztonságossá tétele. Mivel a backend szolgáltatások immár publikusan is elérhetők, szükségessé vált a végpontok megfelelő védelme. Korábban bármely felhasználó szabadon meghívhatta az egyes végpontokat, azonban az új rendszerben kizárólag hitelesített felhasználók férhetnek hozzá az API-khoz érvényes token használatával, amelyet regisztráció során kapnak meg. Emellett jogosultságkezelést is bevezettem, így minden felhasználó kizárólag a saját adatait módosíthatja.

A felhasználói élmény javítása érdekében optimalizáltam a képfeltöltési folyamatot is. A modern okostelefonok alapértelmezetten nagy felbontású képeket készítenek, amelyek feltöltése jelentős időt vehet igénybe, miközben az alkalmazás működéséhez nincs szükség ilyen méretű fájlokra. Ennek megoldására feltöltés előtt automatikus képtömörítést alkalmaztam, amely jelentősen csökkenti a feltöltendő fájlok méretét. Ez gyorsabb feltöltést és ezáltal jobb felhasználói élményt eredményezett.

A kollekciókban szereplő tárgyak közösségi funkciói szintén bővültek. Korábban a felhasználók kizárólag kedvelni tudták egymás tárgyait, illetve ritkasági szempontból értékelhették azokat. Mivel azonban ezt nem tartottam elegendő kommunikációs lehetőségnek, kommentelési funkciót is bevezettem. Ennek köszönhetően a felhasználók immár közvetlenül hozzászólhatnak mások tárgyaihoz, ami erősíti a közösségi interakciókat az alkalmazáson belül.

2.3 Build

A build folyamat során elsőként arra törekedtem, hogy egy egységes és könnyen kezelhető futási környezetet alakítsak ki, amely független az adott operációs rendszertől vagy futtatási környezettől. Miután ezt sikerült megvalósítanom, elkezdtem foglalkozni egy CI/CD pipeline kialakításával annak érdekében, hogy tovább növeljem a projekt minőségét és a fejlesztési folyamat megbízhatóságát.

2.3.1 Backend részek Docker konténerizációja

A Docker-alapú konténerizáció során célom az volt, hogy a korábban Windows környezetben, lokálisan futó backend komponenseket Docker környezetbe migráljam. A backend három fő részből állt: egy PostgreSQL adatbázisból, egy Python alapú FastAPI szolgáltatásból, valamint egy .NET Core Web API szolgáltatásból.

A konténerizáció több szempontból is ésszerű döntésnek bizonyult. Egyrészt a kezdeti konfiguráció - vagyis a Dockerfile-ok és a docker-compose.yml fájlok elkészítése - után a projekt buildelése és indítása jelentősen leegyszerűsödött, gyakorlatilag egyetlen parancs kiadásával elvégezhetővé vált. Másrészt a Docker platformfüggetlen működése lehetővé teszi, hogy a konténerek bármilyen számítógépen ugyanúgy működjenek, amennyiben a Docker telepítve van. Ezzel elkerülhető az a tipikus probléma, amikor egy alkalmazás az egyik gépen megfelelően működik, míg egy másik környezetben különböző konfigurációs eltérések miatt hibák jelentkeznek.

```
//Dockerfile.dotnet
// Fejlesztői környezetben a watch parancsot használjuk, hogy a kód
változásaira automatikusan újrainduljon az alkalmazás
CMD ["dotnet", "watch", "run", "--project",
"Coffer/Coffer.ASPNET/Coffer.ASPNET.csproj", "--urls", "http://0.0.0.0:80"]

//Dockerfile.python
//Fejlesztői környezetben a --reload opciót használjuk, hogy a kód
változásaira azonnal reagáljon a szerver.
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]

version: "3.9"
services:
  api:
    build:
      dockerfile: Coffer/Coffer.ASPNET/Dockerfile.dotnet
    ports:
      - "5141:80"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
    depends_on:
      - db
      - fastapi
    volumes:
      - E:\Homework\Programming\AI\szakdolgozat\data\images:/app/data/images
      - E:\Homework\Programming\AI\szakdolgozat\data\vectors:/app/data/vectors
      - E:\Homework\Programming\AI\szakdolgozat\data\temp:/app/data/temp
  fastapi:
    build:
      dockerfile: Dockerfile.python
    ports:
      - "8000:8000"
    volumes:
      - E:\Homework\Programming\AI\szakdolgozat\data\images:/app/data/images
      - E:\Homework\Programming\AI\szakdolgozat\data\vectors:/app/data/vectors
      - E:\Homework\Programming\AI\szakdolgozat\data\temp:/app/data/temp
  db:
    image: postgres:15
    env_file:
      - .env
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

4. ábra Fejlesztői környezet docker fájlok részletei

```
//Dockerfile.dotnet.prod
// Éles környezetben nincs hot reload, a buildelt DLL-eket futtatjuk
közvetlenül
ENTRYPOINT ["dotnet", "Coffer.ASPNET.dll"]

//Dockerfile.python.prod
// Éles környezetben nincs hot reload, nem kell a --reload flag
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

services:
  api:
    image: ghcr.io/gelomelo1/coffer-api:latest
    ports:
      - "5141:80"
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    depends_on:
      - db
      - fastapi
    volumes:
      - images_data:/app/data/images
      - vectors_data:/app/data/vectors
      - temp_data:/app/data/temp
  fastapi:
    image: ghcr.io/gelomelo1/coffer-python:latest
    expose:
      - "8000"
    volumes:
      - images_data:/app/data/images
      - vectors_data:/app/data/vectors
      - temp_data:/app/data/temp
      - ./models:/app/models
  db:
    image: postgres:15
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "100.85.246.42:5432:5432"

volumes:
  db_data:
  images_data:
  vectors_data:
  temp_data:
```

5. ábra Éles környezet docker fájlok részletei

A migráció során két külön konténerizációs megközelítést alakítottam ki: egyet fejlesztési, egyet pedig éles környezet számára. Mindkét megoldás jelentős előnyökkel járt. A fejlesztői környezet esetében a konténerek biztosítják, hogy egy új fejlesztő gyorsan el tudja kezdeni a munkát anélkül, hogy a helyi konfigurációval vagy buildelési problémákkal kellene foglalkoznia. Az éles környezetre kialakított konténerek pedig nagymértékben leegyszerűsítik a kitelepítési folyamatot, hiszen egy megfelelően konfigurált Nginx vagy Apache webszerver mellett a teljes backend egyetlen parancs segítségével elindítható. A konténerizációhoz tartozó docker-compose konfigurációs fájlokat a 4. ábra és 5. ábra szemlélteti.

A fejlesztői és az éles Dockerfile-ok között a legfontosabb különbség az volt, hogy a fejlesztői verzió hot reload mechanizmust alkalmazott a gyorsabb fejlesztés érdekében. A .NET Core szolgáltatás esetében ez automatikus újrafordítást és újraindítást jelentett módosítás esetén, míg az éles Dockerfile kizárólag a végleges DLL állományokat készítette el. A Python alapú FastAPI szolgáltatás esetében az éles környezetben egyszerűen a FastAPI szerver indult el, hot reload támogatás nélkül.

A két docker-compose konfiguráció között szintén jelentős különbségek voltak. A fejlesztői környezethez tartozó konfiguráció szükség esetén automatikusan buildelte az image-eket, míg az éles környezet már a dokumentum [2.3.2-es](#) pontjában ismertetett GitHub Container Registry-ből töltötte le az előre elkészített image-eket. Emellett a fejlesztői környezetben a konténerek publikálták a szükséges portokat, valamint bind mountokat használtak annak érdekében, hogy a fájlrendszerbeli változások közvetlenül tükröződjenek a konténerekben.

Ezzel szemben az éles környezethez tartozó docker-compose konfiguráció kizárólag a Web API szolgáltatás portját tette közvetlenül elérhetővé. A FastAPI szolgáltatás csak a Docker belső hálózatán keresztül volt hozzáférhető, míg az adatbázis kizárólag egy közös VPN-hálózathoz csatlakozó VPN-tunnelen keresztül volt elérhető, ezzel növelve a rendszer biztonságát.

2.3.2 Docker konténerek CI/CD pipeline-ba integrálása

Az agilitással foglalkozó szakirodalmak egyik fontos alapelve, hogy a legfőbb prioritás az ügyfél igényeinek kielégítése a működő szoftver minél korábbi átadásával, valamint annak folyamatos és rendszeres szállításával. Emellett az agilis szemlélet szerint a fejlesztés előrehaladásának legfontosabb mérőszáma maga a működő szoftver. [1] Ennek a követelménynek szerettem volna megfelelni, ezért elkezdtem kialakítani egy GitHub-alapú CI/CD pipeline-t.

Mivel a projekt verziókezelésére GitHubot használtam, olyan forrásokat kerestem, amelyek részletesen ismertetik a CI/CD megközelítés gyakorlati alkalmazását Git-alapú fejlesztési környezetben. A GitLab dokumentációja részletesen bemutatja a CI/CD szemlélet lényegét, amely röviden úgy foglalható össze, mint a buildelés, tesztelés és telepítés automatizálása a gyorsabb és megbízhatóbb szoftverszállítás érdekében. [2] GitHub környezetben ez elsősorban GitHub Actions workflow-k és branch protection szabályok alkalmazását jelenti.

```
name: Build .NET + Python
on:
  push:
    branches:
      - development
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # Checkout code
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Build .NET solution
        run: dotnet build ./src/Coffer.sln
      - name: Python compile check
        run: python -m compileall -l ./src/Coffer/Coffer.PythonService
      # Setup Docker build system (enables caching)
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

# .NET BUILD
      - name: Build .NET Docker image
        uses: docker/build-push-action@v5
        with:
          context: ./src
          file: ./src/Coffer/Coffer.ASPNET/Dockerfile.dotnet

# PYTHON BUILD
      - name: Build Python Docker image
        uses: docker/build-push-action@v5
        with:
          context: ./src/Coffer/Coffer.PythonService
          file: ./src/Coffer/Coffer.PythonService/Dockerfile.python
```

6. ábra Fejlesztői környezet GitHub Actions workflow részletei

A projekt során három GitHub Actions workflow-t készítettem a CI/CD folyamat támogatására. Az első workflow részleteit a 6. ábra mutatja. Ez a szkript a development branchre történő pusholáskor fut le, és több ellenőrzést végez: buildeli a .NET Core backend szolgáltatást, szintaktikai ellenőrzést futtat a Python-alapú szolgáltatáson, valamint ellenőrzi, hogy a Docker image-ek sikeresen buildelhetők-e. Ezt a pipeline-szakaszt azért hoztam létre, hogy a fejlesztők minden push után azonnali visszajelzést kapjanak arról, hogy a projekt buildelhető-e, és hiba esetén gyorsan javítani tudják a problémákat.

```
name: Build & Push .NET + Python Production Images
on:
  pull_request:
    branches:
      - main
permissions:
  contents: read
  packages: write
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      # 1. Checkout source
      - name: Checkout repository
        uses: actions/checkout@v4
      # 2. Login to GHCR
      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      # 3. Enable Docker Buildx
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      # 4. BUILD & PUSH .NET API
      - name: Build and push .NET API
        uses: docker/build-push-action@v5
        with:
          tags: |
            ghcr.io/${ github.repository_owner }/coffer-api:latest
            ghcr.io/${ github.repository_owner }/coffer-api:${ github.sha }
      # 5. BUILD & PUSH PYTHON FASTAPI
      - name: Build and push Python API
        uses: docker/build-push-action@v5
        with:
          tags: |
            ghcr.io/${ github.repository_owner }/coffer-python:latest
            ghcr.io/${ github.repository_owner }/coffer-python:${ github.sha }
      - name: Build and push Python API
        uses: docker/build-push-action@v5
        with:
          tags: |
            ghcr.io/${ github.repository_owner }/coffer-python:latest
            ghcr.io/${ github.repository_owner }/coffer-python:${ github.sha }
```

7. ábra Éles környezet GitHub Actions workflow részletei

A második workflow részleteit a 7. ábra szemlélteti. Ez a workflow nagyrészt hasonló a development környezethez tartozó verzióhoz, azonban pull request létrehozásakor fut le, és már az éles környezethez készített Dockerfile-okat használja. A sikeresen buildelt image-eket ezt követően feltölti a GitHub Container Registry-be. Ezáltal az éles telepítés során nem szükséges újra buildelni az alkalmazást, elegendő a már elkészített image-ek letöltése.

```
name: Deploy to VPS

on:
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.VPS_HOST }
          username: root
          key: ${ secrets.VPS_SSH_KEY }
          script:
```

8. ábra Deploy GitHub Actions workflow részletei

A harmadik workflow maga a telepítési folyamat, amelynek részleteit a 8. ábra mutatja. Ez a workflow nem automatikusan, esemény hatására indul el, hanem manuálisan indítható a fejlesztő által. A workflow-t futtató GitHub Actions runner SSH-kapcsolaton keresztül csatlakozik a VPS szerverhez, majd végrehajtja a szükséges telepítési parancsokat. A folyamat része a Docker image-ek letöltése a GitHub Container Registry-ből, majd a konténerek elindítása.

The screenshot shows the 'Protect matching branches' configuration page in GitHub. It lists several protection rules that are enabled:

- Require a pull request before merging**: When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.
- Require approvals**: When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged. The 'Required number of approvals before merging' is set to 1.
- Dismiss stale pull request approvals when new commits are pushed**: New reviewable commits pushed to a matching branch will dismiss pull request review approvals.
- Require review from Code Owners**: Require an approved review in pull requests including files with a designated code owner.
- Require approval of the most recent reviewable push**: Whether the most recent reviewable push must be approved by someone other than the person who pushed it.
- Require status checks to pass before merging**: Choose which [status checks](#) must pass before branches can be merged into a branch that matches this rule. When enabled, commits must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule after status checks have passed.
- Require branches to be up to date before merging**: This ensures pull requests targeting a matching branch have been tested with the latest code. This setting will not take effect unless at least one status check is enabled (see below).

Below these rules, there is a search bar for status checks in the last week for this repository. A table titled 'Status checks that are required' shows one check: 'build-and-push' with a 'GitHub Actions' icon and a close button.

At the bottom, two more rules are listed:

- Require conversation resolution before merging**: When enabled, all conversations on code must be resolved before a pull request can be merged into a branch that matches this rule. [Learn more about requiring conversation completion before merging.](#)
- Require signed commits**: Commits pushed to matching branches must have verified signatures.

9. ábra Release Ready Mainline branch protection szabályai

A teljes folyamatot a 9. ábra által mutatott branch protection konfiguráció fogja össze. A projekt során a Release Ready Mainline szemléletet követtem, amelynek lényege, hogy a fejlesztés külön development brancheken történik, míg a main branch kizárólag stabil, kiadásra kész állapotú kódot tartalmazhat. A main branchre közvetlen pusholás nem engedélyezett; módosítás kizárólag pull requesten keresztül kerülhet be. Emellett a merge-hez legalább egy jóváhagyás szükséges, valamint minden kötelező build- és tesztfolyamatnak sikeresen le kell futnia. További feltétel volt, hogy csak aláírt (signed) commitok kerülhessenek a branch történetébe, biztosítva ezzel a commitok hitelességét és eredetének ellenőrizhetőségét.

Ennek eredményeként megvalósult a folyamatos szállítás (continuous delivery) alapelve, hiszen a main branch minden időpillanatban egy működőképes, telepíthető projektverziót tartalmaz.

2.4 Deploy

A deploy szakaszban kellett eldöntenem, hogy a backendet tartalmazó konténereket hova hostolom ki.

2.4.1 A megfelelő VPS kiválasztása

Ebben a szakaszban két lehetséges irány közül választhattam. Az egyik lehetőség a magasabb szintű, Platform as a Service (PaaS) megközelítés volt - például AWS vagy Azure használata -, ahol a szolgáltató számos konfigurációs feladatot automatizál, illetve támogatást nyújt az infrastruktúra kezelésében. A másik lehetőség az Infrastructure as a Service (IaaS) modell volt, amelyet különböző VPS-szolgáltatók kínálnak. Ebben az esetben lényegében egy

üres virtuális szervert kap a felhasználó, amelynek teljes konfigurációját saját magának kell elvégeznie.

Végül az IaaS-alapú VPS megoldás mellett döntöttem, mivel szerettem volna Linux-alapú szerverüzemeltetési és szkriptelési tapasztalatot szerezni, emellett költséghatékony megoldást kerestem. A következő lépés a megfelelő VPS-szolgáltató kiválasztása volt. Elsődleges szempontként a teljesítmény-ár arányt határoztam meg, míg a rendelkezésre álló tárhely másodlagos szerepet kapott. Bár a .NET Core backend nem végez kifejezetten számításigényes műveleteket, a szervernek egyszerre kellett futtatnia egy PostgreSQL adatbázist, egy .NET Core Web API szolgáltatást, valamint egy Python-alapú AI szolgáltatást is, így a minimálisnál valamivel erősebb hardverre volt szükség. A tárhelyigény ezzel szemben nem volt jelentős, mivel az alkalmazás tömöríti a feltöltött képeket, így azok viszonylag kevés helyet foglalnak.

A választásban nagy segítséget nyújtott egy Medium-cikk, [3] amely összehasonlította a legkedvezőbb árú hostingmegoldásokat és VPS-szolgáltatókat. Végül a Hetzner szolgáltató mellett döntöttem, mivel a cikk szerint ők kínálták a legjobb teljesítmény-ár arányú szerverkonfigurációkat, emellett a webes kezelőfelületük is felhasználóbarát volt.

A választásom a CPX22 csomagra esett, amely 2 virtuális CPU-magot, 4 GB RAM-ot, 80 GB SSD tárhelyet, 20 TB havi adatforgalmat, valamint publikus IPv4- és IPv6-címet biztosított havi 10,15 € díjért. A beszámoló készítésekor ez körülbelül 3606 forintnak felelt meg. A szolgáltatással kapcsolatban eddig kizárólag pozitív tapasztalataim voltak: használat közben nem tapasztaltam teljesítményproblémát vagy kiesést. Emellett a szolgáltató számos további funkciót is biztosít, például szerver-átméretezést vagy load balancing megoldásokat, amelyek lehetőséget adnak a rendszer későbbi bővítésére és skálázására.

2.4.2 Backend konténerek telepítése a VPS -re

A konténerek telepítése a kezdeti szerverkonfiguráció után már viszonylag egyszerű folyamat volt. A kezdeti konfiguráció során azokat az alapvető lépéseket végeztem el, amelyeket általában egy üres VPS-szerver hosting célú használata előtt szükséges beállítani. Elsőként konfiguráltam az SSH-hozzáférést annak érdekében, hogy ne kizárólag a Hetzner webes felületéről tudjak csatlakozni a szerverhez, hanem közvetlenül a saját gépemről is.

Ezt követően a tűzfalbeállításokat konfiguráltam: minden bejövő forgalmat letiltottam, kivéve a működéshez szükséges SSH, HTTP és HTTPS portokat. Emellett beállítottam az NGINX webszervert is, amely reverse proxyként működik, és ezen keresztül éri el a mobilalkalmazás az API-végpontokat.

A publikus elérés biztosításához a DuckDNS szolgáltatás segítségével szereztem egy ingyenes domainnevet. Ezt követően a Certbot használatával TLS-tanúsítványt állítottam ki a domainhez, így az alkalmazás és a webszerver közötti kommunikáció már HTTPS titkosítással történik.

A konfiguráció elvégzése után az új verziók telepítése jelentősen leegyszerűsödött. Egy új kiadás publikálásához elegendő elindítani a [2.3.2-es](#) fejezetben ismertetett GitHub Actions deploy workflow-t, amely automatikusan végrehajtja a szükséges telepítési lépéseket a VPS szerveren.

2.5 Összefoglalás

Az önálló laboratórium során modelleztem egy agilis szoftverfejlesztési életciklus több fontos szakaszát. Foglalkoztam a projekt feladatkövetési rendszerének fejlesztésével, az agilis módon felmerülő új funkciók implementálásával, a backend komponensek konténerizálásával, valamint az erre épülő CI/CD pipeline kialakításával. Emellett kiválasztottam egy, a projekt igényeinek megfelelő és költséghatékony VPS-szolgáltatót, majd megvalósítottam a backend,

szerverre történő telepítését is.

A projekt jelenlegi állapotát az év eleji verzióval összehasonlítva egyértelmű fejlődés figyelhető meg szinte minden területen. A rendszer minősége jelentősen javult, emellett a különböző automatizációs és támogató megoldásoknak köszönhetően a fejlesztési folyamat is kényelmesebbé és hatékonyabbá vált. Személyes célokat is sikerült elérnem, hiszen a backend szolgáltatásokat sikeresen kitelepítettem egy nyilvánosan elérhető szerverre, amelyet a mobilalkalmazás már biztonságos HTTPS-kapcsolaton keresztül bárholnan képes elérni.

Továbbfejlesztési irányként elsősorban a frontend fejlesztői környezet konténerizálását, valamint egy automatikus APK-build folyamat kialakítását tartom fontosnak, amely pull request létrehozásakor automatikusan elkészítené az alkalmazás telepíthető verzióját. Ez jelentős segítséget nyújtana az új eszközökön történő fejlesztés és tesztelés során, mivel a tapasztalataim alapján a backend környezet megfelelő konfigurálása után leginkább a frontend fejlesztői környezet beállítása okozott problémát.

A frontend oldalon React Native technológiát használok Expo bundlerrel, amely számos hasznos funkciót biztosít, ugyanakkor jóval összetettebb és rugalmasabban konfigurálható környezetet jelent, mint a backend oldali .NET Core szolgáltatás. Emiatt a fejlesztői környezet beállítása több ponton is hibalehetőséget hordoz magában. Az Expo és a különböző SDK-verziók kezelése tovább növeli a rendszer komplexitását, ezért egy megfelelően működő frontend konténerkörnyezet kialakítása összetett feladat lenne. Ugyanakkor ennek sikeres megvalósítása jelentősen gyorsítaná és egyszerűsítaná a fejlesztési folyamatot.

Összességében az önálló laboratórium rendkívül hasznos tapasztalatot jelentett számomra. Úgy érzem, olyan területeken szereztem értékes gyakorlati tudást - különösen az agilis fejlesztés, a DevOps és az infrastruktúra-kezelés területén -, amelyek sok fejlesztő számára nehézséget jelenthetnek. Ezek a tapasztalatok várhatóan a jövőbeni projektjeim során is jelentős segítséget nyújtanak majd.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

A tanulmányozott irodalom jegyzéke:

- [1] A. Alliance, „AGILE ESSENTIALS,” [Online]. Available: <https://agilealliance.org/agile101/12-principles-behind-the-agile-manifesto/>. [Hozzáférés dátuma: 10 május 2026].
- [2] A. V., „Best Cheap VPS Hosting in 2026: Under \$5/Month Picks That Work,” 15 február 2026. [Online]. Available: <https://medium.com/@velinxs/best-cheap-vps-hosting-in-2026-under-5-month-picks-that-work-719810d47ba9>. [Hozzáférés dátuma: 10 május 2026].
- [3] GitLab, „What is CI/CD?,” [Online]. Available: <https://about.gitlab.com/topics/ci-cd/>. [Hozzáférés dátuma: 10 május 2026].